SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Module 5 Inference in FOL - Resolution

Dr. Nirmala Baloorkar
Assistant Professor
Department of Computer Engineering KJSSE

Outline

- Resolution
- Clausal Form
- CNF

Universal Instantiation (UI)

Universal Instantiation allows us to take a universally true statement and apply it to a **specific object**.

Rule (Formal)

- If a property is true for **all objects**, then it is true for **any particular object**.

$$\forall x\ P(x) \Rightarrow P(c) \quad \text{(where c is a constant)}$$

$$\frac{\forall v \alpha}{\mathrm{SUBST}(\{v/g\}\alpha)}, \text{ where } g \text{ is a ground term.}$$

Example

- All humans are mortal.
- **FOL:**
- $\forall x\ \mathrm{Human}(x) \rightarrow \mathrm{Mortal}(x)$
- **Apply Universal Instantiation (x = Socrates):**
- $\mathrm{Human}(\mathrm{Socrates}) \rightarrow \mathrm{Mortal}(\mathrm{Socrates})$

Existential Instantiation (EI)

Existential Instantiation allows us to take a statement saying **something exists** and replace it with a **new constant** representing that object.

Rule

$$\frac{\exists v \alpha}{\mathrm{SUBST}(\{v/k\}\alpha)}$$, where k is a new constant.

∃x P(x)⇒P(K) (where K is a **new constant**, never used before)

Example

- There exists a student who is intelligent.
- **FOL:**
- ∃x Intelligent(x)
- **Apply Existential Instantiation:**
- Intelligent(S1)

Unification

- **Unification** is the process of making **two logical expressions identical** by finding suitable substitutions for variables.

Example

- Likes(x,IceCream), Likes(Rahul,IceCream)
- **Substitution:** x=Rahul
- Parent(x,y), Parent(Ram,Shyam)
- **Substitution:** x=Ram, y=Shyamx = Ram

Lifting

- **Lifting** is performing inference **directly on predicates with variables**, instead of converting everything into ground (constant-based) sentences.
- Example
- If P(A) and ∀xP(x)-> Q(x) , lifting uses unification to match P(A) with P(x) by setting x=A , allowing you to infer Q(A) directly.
-

What is Resolution?

- Resolution is a method of **theorem proving** that involves **constructing refutation proofs**, or **proofs by contradiction**.
- It was created in **1965 by mathematician John Alan Robinson**.
- The method is based on the principle of **proof by contradiction** and **uses unification** to simplify the process.
- Resolution is a single inference rule that can work efficiently on either the **conjunctive normal form (CNF) or the clausal form**.

Steps in Resolution by Refutation

- **Convert to Conjunctive Normal Form (CNF)**:
  - Convert all the statements (including the negation of the statement to be proved) into CNF.
- **Negate the Statement**:
  - Negate the statement that you want to prove.
- **Combine All Statements**:
  - Combine all the given statements and the negation of the statement to be proved into a single set of clauses.

Steps in Resolution by Refutation

- **Apply Resolution Rule**:
  - Apply the resolution rule to the set of clauses. The resolution rule states that if you have two clauses, one containing a literal and the other containing its negation, you can combine them to form a new clause that contains all the literals of the two clauses except the resolved literal and its negation.
- **Derive Empty Clause**:
  - Continue applying the resolution rule until you derive an empty clause (a contradiction).
- **Conclusion**:
  - Since the negation of the statement leads to a contradiction, the original statement must be true.

Steps to Convert to Clausal Form

- **Eliminate Implications**:
  - Replace implications ($\rightarrow$) with their equivalent forms using disjunctions and negations.
    - $P \rightarrow QP$ becomes $\neg P \vee Q$.
- **Move Negations Inward**:
  - Use De Morgan's laws to move negations inward and eliminate double negations.
    - $\neg(\neg P)$ becomes P.
    - $\neg(P \wedge Q)$ becomes $\neg P \vee \neg Q$.
    - $\neg(P \vee Q)$ becomes $\neg P \wedge \neg Q$

Steps to Convert to Clausal Form

- **Distribute Disjunctions over Conjunctions**:
  - Apply distributive laws to ensure that the formula is a conjunction of disjunctions.
    - $P \vee (Q \wedge R)$ becomes $(P \vee Q) \wedge (P \vee R)$
- **Flatten Nested Conjunctions and Disjunctions**:
  - Ensure that the formula is a flat conjunction of disjunctions.

Example – Clausal Form

$(A \wedge B) \vee C$

To convert this to clausal form:

- **Distribute the disjunction over the conjunction**:

$(A \vee C) \wedge (B \vee C)$

- **Express as a set of clauses**:

  Clause 1: $A \vee C$

  Clause 2: $B \vee C$

Example 1

Translate the following set of sentences into CNF. Use resolution to prove Raja is angry.

1. Rimi is hungry
2. if Rimi is hungry, she barks.
3. if Rimi is barking then Raja is angry.

Example 1

FOL

1. Rimi is hungry

   Hungry(Rimi)

2. if Rimi is hungry, she barks.

   Hungry(Rimi) ? Bark(Rimi)

3. if Rimi is barking then Raja is angry.

   Bark(Rimi) ? Angry(Raja)

CNF

1. Hungry(Rimi)
2. Hungry(Rimi) ? Bark(Rimi)

   ~ Hungry(Rimi) V Bark(Rimi)

3. Bark(Rimi) ? Angry(Raja)

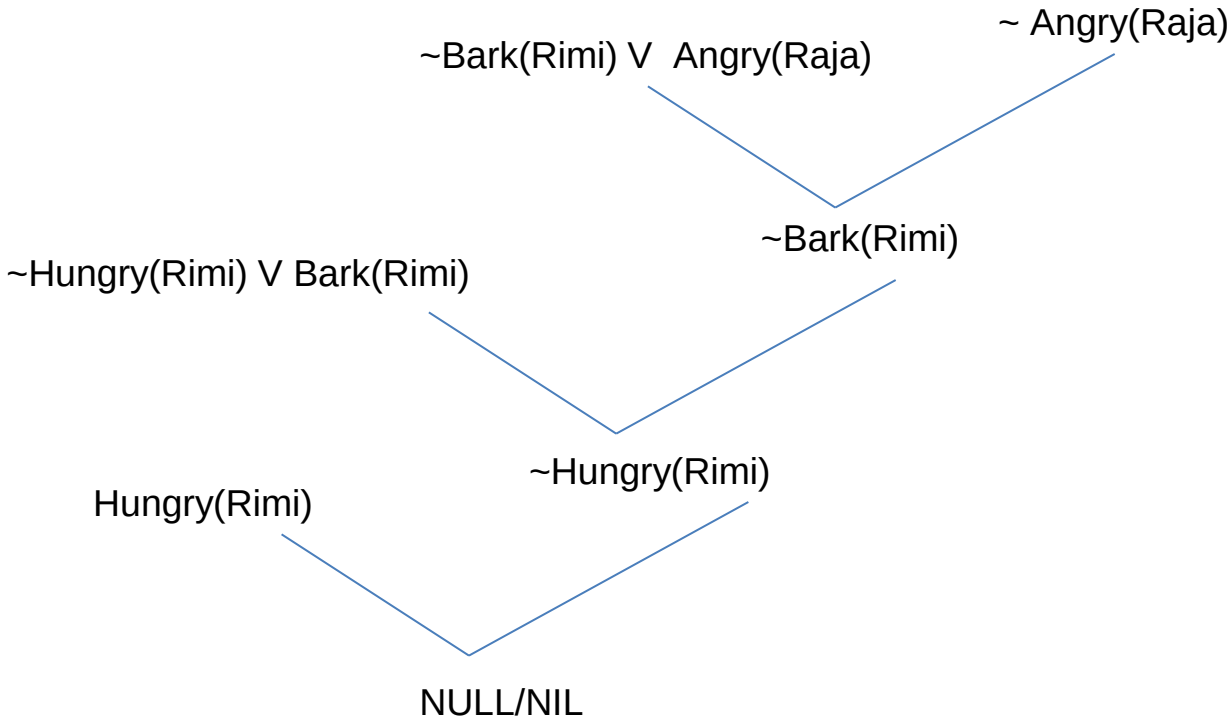
   ~Bark(Rimi) V Angry(Raja)

Resolution

- Negate the goal

~ Angry(Raja)

1. Hungry(Rimi)
2. ~ Hungry(Rimi) V Bark(Rimi)
3. ~Bark(Rimi) V Angry(Raja)



Q3. (a) Consider the following axioms: [4+4+4]

All people who are graduating are happy.
All happy people smile.
Someone is graduating.

(i) Represent these axioms in first order predicate logic.
(ii) Convert each formula to clause form
(iii) Prove that **"Is someone smiling?"** using resolution technique. Draw the resolution tree.

Example 1 knowledge base

- The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American.

- Prove that Col. West is a criminal

Example 1 knowledge base (continued)

… it is a crime for an American to sell weapons to hostile nations:

$American(x) \wedge Weapon(y) \wedge Sells(x,y,z) \wedge Hostile(z) \Rightarrow Criminal(x)$ …1

Nono … has some missiles, i.e.,

$\exists x\ Owns(Nono,x) \wedge Missile(x)$:

$Owns(Nono,M_1)$ and $Missile(M_1)$ …2

… all of its missiles were sold to it by Colonel West

$Missile(x) \wedge Owns(Nono,x) \Rightarrow Sells(West,x,Nono)$ …3

Example 1 knowledge base (continued)

… Missiles are weapons:

Missile(x) ⇒ Weapon(x) …4

An enemy of America counts as "hostile“:

Enemy(x,America) ⇒ Hostile(x) …5

West, who is American …

American(West) …6

The country Nono, an enemy of America …

Enemy(Nono,America) …7

1. American(x) ∧ Weapon(y) ∧ Sells(x,y,z) ∧ Hostile(z) ⇒ Criminal(x)
2. Owns(Nono,M_1) ∧ Missile(M_1)
3. Missile(x) ∧ Owns(Nono,x) ⇒ Sells(West,x,Nono)
4. Missile(x) ⇒ Weapon(x)
5. Enemy(x,America) ⇒ Hostile(x)
6. American(West)
7. Enemy(Nono,America)
8. Goal : Criminal (West) for proving negate the goal as ~Criminal(West)

Clausal Form

1. ~American(x) v ~Weapon(y) v ~Sells(x,y,z) v ~Hostile(z) v Criminal(x)
2. Owns(Nono,M_1) ∧ Missile(M_1)
3. ~Missile(x) v ~Owns(Nono,x) v Sells(West, x, Nono)
4. ~Missile(x) v Weapon(x)
5. ~Enemy(x,America) v Hostile(x)
6. Americam(West)
7. Enemy(Nono, America)
8. ~Criminal(West)

1. ~American(x) v ~Weapon(y) v ~Sells(x,y,z) v ~Hostile(z) v Criminal(x)
2. Owns(Nono,M_1) ∧ Missile(M_1)
3. ~Missile(x) v ~Owns(Nono,x) v Sells(West, x, Nono)
4. ~Missile(x) v Weapon(x)
5. ~Enemy(x,America) v Hostile(x)
6. Americam(West)
7. Enemy(Nono, America)
8. ~Criminal(West)

~Criminal(West)

~American(x) v ~Weapon(y) v ~Sells(x,y,z) v ~Hostile(z) v Criminal(x)

x/West

~American(West) v ~Weapon(y) v ~Sells(West,y,z) v ~Hostile(z)

American(West)

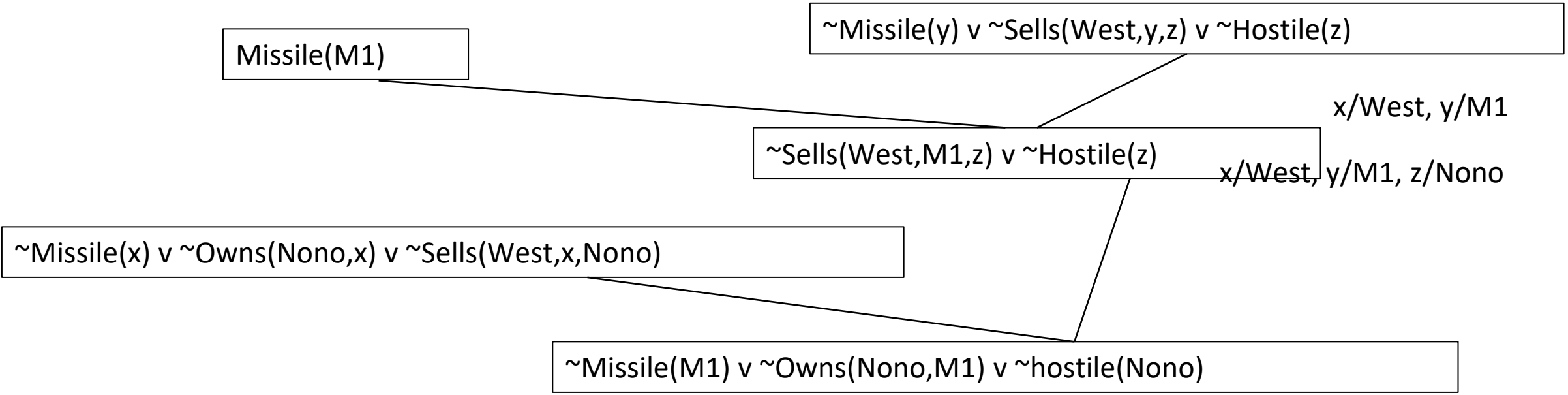
~Weapon(y) v ~Sells(West,y,z) v ~Hostile(z)

x/West

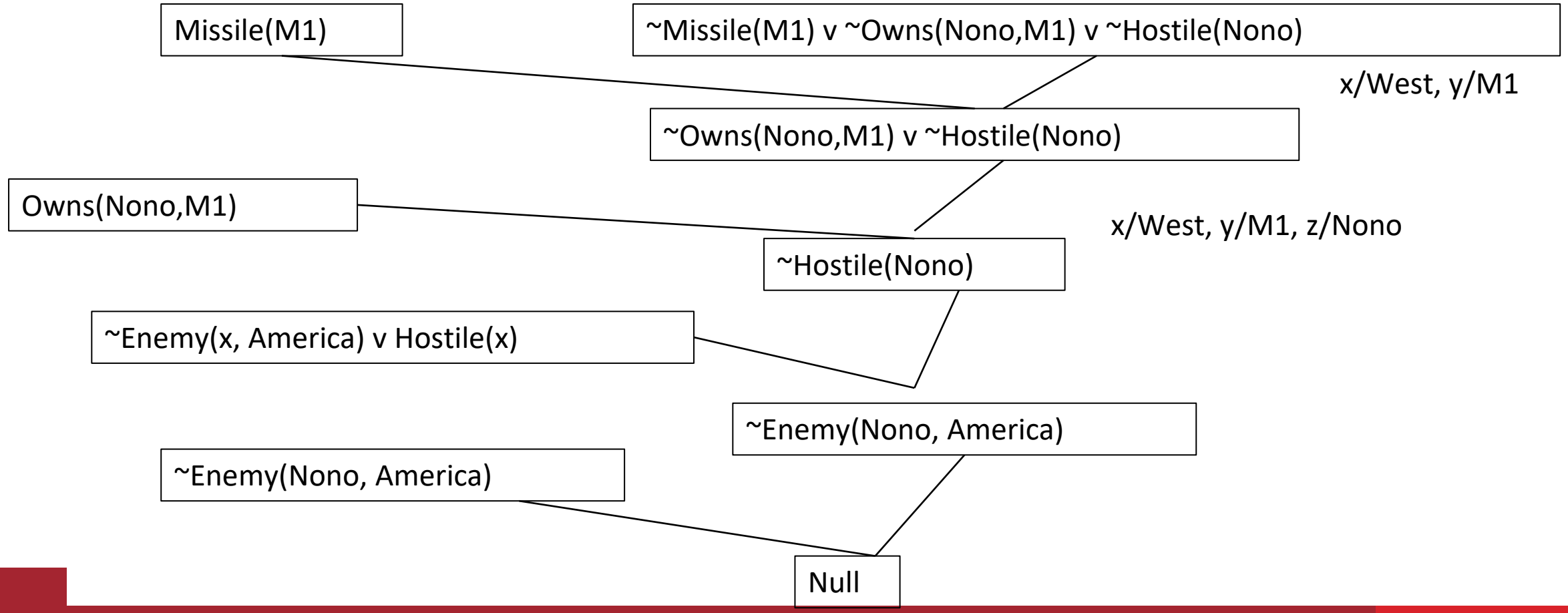
~Missile(x) v Weapon(x)

~Missile(y) v ~Sells(West,y,z) v ~Hostile(z)

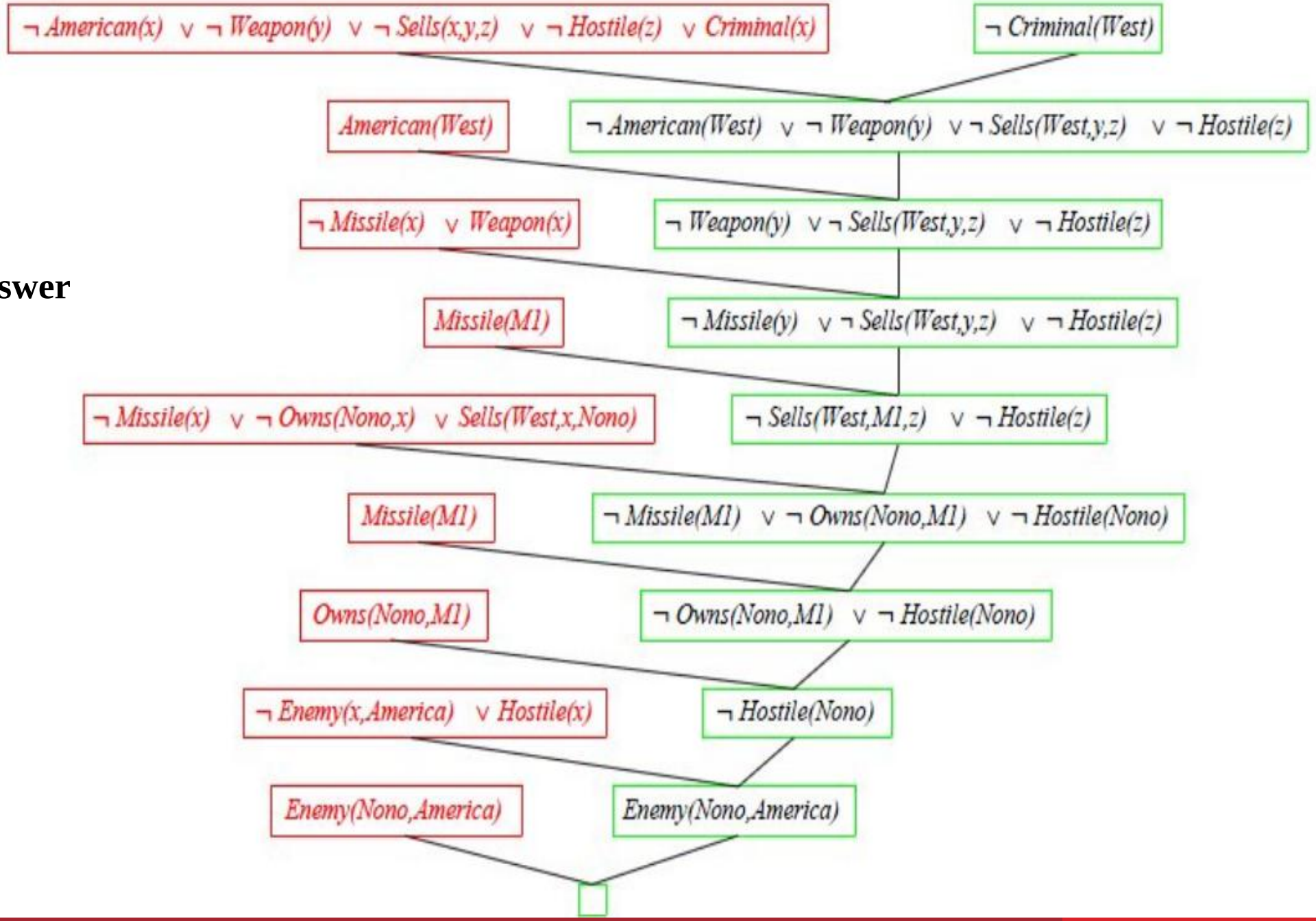
1. ~American(x) v ~Weapon(y) v ~Sells(x,y,z) v ~Hostile(z) v Criminal(x)
2. Owns(Nono,M_1) ∧ Missile(M_1)
3. ~Missile(x) v ~Owns(Nono,x) v Sells(West, x, Nono)
4. ~Missile(x) v Weapon(x)
5. ~Enemy(x,America) v Hostile(x)
6. Americam(West)
7. Enemy(Nono, America)
8. ~Criminal(West)



1. ~American(x) v ~Weapon(y) v ~Sells(x,y,z) v ~Hostile(z) v Criminal(x)
2. Owns(Nono,M_1) ∧ Missile(M_1)
3. ~Missile(x) v ~Owns(Nono,x) v Sells(West, x, Nono)
4. ~Missile(x) v Weapon(x)
5. ~Enemy(x,America) v Hostile(x)
6. American(West)
7. Enemy(Nono, America)
8. ~Criminal(West)



Example 2 Final Answer



Example 2

- If the maid stole the jewelry, then the butler wasn't guilty. Either the maid stole the jewelry or she milked the cows. If the maid milked the cows, then the butler got the cream.
- Goal: Therefore, if the butler was guilty, then he got the cream.

G(M) : The maid stole jewelry

G(B) : Butler was guilty

Cows(M) : Maid milked cow

Cream(B) : Butler got cream.

G(M) : The maid stole jewelry
G(B) : Butler was guilty
Cows(M) : Maid milked cow
Cream(B) : Butler got cream.

If the maid stole the jewelry, then the butler wasn't guilty.

G(M) -> ~G(B) …1

Either the maid stole the jewelry or she milked the cows.

G(M) v Cows(M) …2

If the maid milked the cows, then the butler got the cream

Cows(M) -> Cream(B) …3

Goal: Therefore, if the butler was guilty, then he got the cream.

G(B) -> Cream(B)

But in resolution we negate the goal/proving statement.

~ (G(B) -> Cream(B)) …4

G(M) : The maid stole jewelry
G(B) : Butler was guilty
Cows(M) : Maid milked cow
Cream(B) : Butler got cream.

G(M) -> ~G(B) …1	Eliminate implication
G(M) v Cows(M) …2	
Cows(M) -> Cream(B) …3	Eliminate implication
~ (G(B) -> Cream(B)) …4	Eliminate implication

Clausal Form:

1. ~G(M) v ~G(B)
2. G(M) v Cows(M)
3. ~Cows(M) v Cream(B)

~ (G(B) -> Cream(B)) …4

- After eliminating implication

$\neg(\neg G(B) \vee Cream(B))$

- Move Negation Inwards (De Morgan's Law)

$\neg(A \vee B) \equiv (\neg A \wedge \neg B)$

Apply this rule:

$\neg(\neg G(B) \vee Cream(B))$

$\Longrightarrow \neg(\neg G(B)) \wedge \neg Cream(B)$

$\neg(\neg P) \equiv P$ So,

$\neg(\neg G(B)) = G(B)$

- Final result:

$G(B) \wedge \neg Cream(B)$

- **Rule (AND Elimination)**

$A \wedge B \Rightarrow A, B$

- $G(B) \wedge \neg Cream(B)$
- two separate clauses:

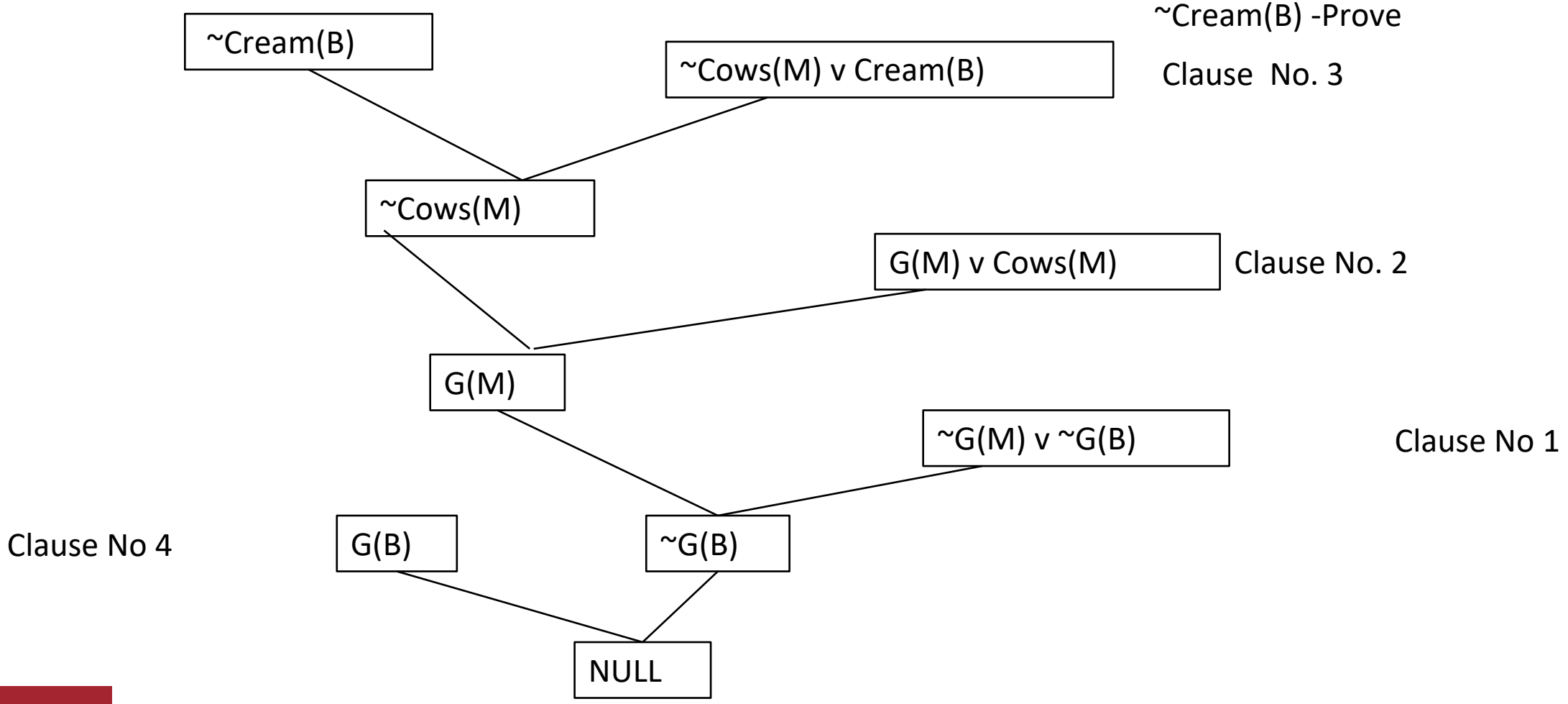
$G(B)$

$\neg Cream(B)$

KB

1. ~G(M) v ~G(B)
2. G(M) v Cows(M)
3. ~Cows(M) v Cream(B)
4. G(B)

~Cream(B) -Prove



Conjunctive Normal Form (CNF) for First Order Logic

1. Eliminate Biconditional ⇔

   $P \Leftrightarrow Q \equiv (P \rightarrow Q) \wedge (Q \rightarrow P)$

2. Eliminate Implication ->

   $P \rightarrow Q \equiv (\neg P \vee Q)$

3. Move negation inwards and reduce scope of “not” sign

   $\sim(\sim P) \equiv P$ double negation

   $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$ de Morgan

   $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ de Morgan

   $\neg\forall x\, P(x) \equiv \exists\, \neg P(x)$ with quantifiers

   $\neg \exists\, x\, P(x) \equiv \forall\, \neg P(x)$ with quantifiers

Conjunctive Normal Form (CNF) for First Order Logic (continued)

4. **Standardize variables**: Rename variables so that each different variable in the set of wffs has a different symbol. i.e. if the sentences use the same variable name twice, change the name of the one of the variables.

$$(\exists x\ P(x)) \vee (\exists x\ Q(x)) \equiv (\exists x\ P(x)) \vee (\exists y\ Q(y))$$

$$\forall x\ [P(x) \rightarrow R(x)] \vee [\exists x(Px)] \equiv [\forall x\ [P(x) \rightarrow R(x)]] \vee [\exists y\ P(y)]$$

$$\neg\forall x\ P(x) \equiv \exists\ \neg P(x)$$

Conjunctive Normal Form (CNF) for First Order Logic (continued)

5. **Skolemization:** Eliminate existential quantifiers and replace existentially quantized variable by **skolem constant or skolem functions** as appropriate.

   $\exists x: P(x) \equiv P(C)$ Skolem constant

   $\forall x: \exists y\ P(x, y) \equiv \forall x: P(x, f(x))$ Skolem function

6. Drop universal quantifiers.

   $\mathbf{\forall x: P(x) \equiv P(x)}$

7. Use distributive laws and equivalence rules of predicate logic to transform to CNF.

Example 3 : CNF

"everyone who loves all animals is loved by someone"

$\forall x: [\ \forall y: animal(y) \Rightarrow loves(x, y)\ ]$

everyone who loves all animals

$\exists y: loves(y, x)$ there exist someone y and y loves x

Thus the predicate sentence is:

$\forall x: [\ [\forall y: animal(y) \Rightarrow loves(x, y)\ ] \Rightarrow [\ \exists y: loves(y, x)\ ]\ ]$

Example 3 : CNF (continued)

1. Eliminate implications

$\forall x\ [\forall y\ \neg Animal(y) \vee Loves(x,y)] \Rightarrow [\exists y\ Loves(y,x)]$ …inner implication

$\forall x\ [\neg\forall y\ \neg Animal(y) \vee Loves(x,y)] \vee [\exists y\ Loves(y,x)]$ …outer implication

2. Move ¬ inwards: $\neg\forall x\ p \equiv \exists x\ \neg p$, $\neg\ \exists x\ p \equiv \forall x\ \neg p$

$\forall x\ [\neg\forall y\ \neg Animal(y) \vee Loves(x,y)] \vee [\exists y\ Loves(y,x)]$

$\forall x\ [\exists y\ \neg(\neg Animal(y) \vee Loves(x,y))] \vee [\exists y\ Loves(y,x)]$

$\forall x\ [\exists y\ \neg\neg Animal(y) \wedge \neg Loves(x,y)] \vee [\exists y\ Loves(y,x)]$

$\forall x\ [\exists y\ Animal(y) \wedge \neg Loves(x,y)] \vee [\exists y\ Loves(y,x)]$

Example 3 : CNF (continued)

3. Standardize variables: each quantifier should use a different one

   $\forall x\ [\exists y\ Animal(y) \land \neg Loves(x,y)] \lor [\exists z\ Loves(z,x)]$

4. Skolemize: a more general form of existential instantiation. Each existential variable is replaced by a Skolem function of the enclosing universally quantified variables:

   $\forall x\ [Animal(F(x)) \land \neg Loves(x,F(x))] \lor Loves(G(z),x)$

Example 3 : CNF (continued)

5. Drop universal quantifiers:

$[Animal(F(x)) \land \neg Loves(x,F(x))] \lor Loves(G(z),x)$

6. Distribute $\lor$ over $\land$:

$[Animal(F(x)) \lor Loves(G(z),x)] \land$
$[\neg Loves(x,F(x)) \lor Loves(G(z),x)]$

Steps for Resolution

- Conversion of facts into first-order logic.
- Convert FOL statements into CNF
- Negate the statement which needs to prove (proof by contradiction)
- Draw resolution graph (unification).

Example 4

- All people who are Graduating are Happy.
- All Happy people smile.
- Someone is Graduating.

- Prove – Is someone Smiling?

Example 4 (continued)

- Predicate

  Graduating(x) : x is Graduating

  Happy(x) : x is Happy

  Smile(x) : x is Smiling

- In FOL

  ∀x Graduating(x) -> Happy(x)

  ∀x Happy(x) -> Smiling(x)

  ∃x Graduating(x)

- All people who are Graduating are Happy.
- All Happy people smile.
- Someone is Graduating.

- Prove – Is someone Smiling?

  Goal: ∃x Smiling(x)

  Negate this ¬∃x Smiling(x)

  Convert in CNF.

Example 4 (continued)

- KB

1. $\forall x$ Graduating(x) $\Rightarrow$ Happy(x)
2. $\forall x$ Happy(x) $\Rightarrow$ Smiling(x)
3 $\exists x$ Graduating(x)
4. $\neg\exists x$ Smiling(x) Goal

- Step 1. Eliminate $\Rightarrow$

1. $\forall x$ $\neg$Graduating(x) $\vee$ Happy(x)
2. $\forall x$ $\neg$Happy(x) $\vee$ Smiling(x)
3. $\exists x$ Graduating(x)
4. $\neg\exists x$ Smiling(x)

Example 4 (continued)

1. $\forall x\ \neg Graduating(x) \vee Happy(x)$
2. $\forall x\ \neg Happy(x) \vee Smiling(x)$
3. $\exists x\ Graduating(x)$
4. $\neg\exists x\ Smiling(x)$

Step 2. Move ¬ inwards.

1. $\forall x\ \neg Graduating(x) \vee Happy(x)$
2. $\forall x\ \neg Happy(x) \vee Smiling(x)$
3. $\exists x\ Graduating(x)$
4. $\forall x\ \neg Smiling(x)$

Example 4 (continued)

1. $\forall x\ \neg Graduating(x) \vee Happy(x)$
2. $\forall x\ \neg Happy(x) \vee Smiling(x)$
3. $\exists x\ Graduating(x)$
4. $\forall x\ \neg Smiling(x)$

Step 3. Standardize variables apart.

1. $\forall x\ \neg Graduating(x) \vee Happy(x)$
2. $\forall y\ \neg Happy(y) \vee Smiling(y)$
3. $\exists z\ Graduating(z)$
4. $\forall w\ \neg Smiling(w)$

Example 4 (continued)

1. ∀x ¬Graduating(x) ∨ Happy(x)
2. ∀y ¬Happy(y) ∨ Smiling(y)
3. ∃z Graduating(z)
4. ∀w ¬Smiling(w)

Step 4. Skolemize.

1. ∀x ¬Graduating(x) ∨ Happy(x)
2. ∀y ¬Happy(y) ∨ Smiling(y)
3. Graduating(NoName1)
4. ∀w ¬Smiling(w)

Example 4 (continued)

1. ∀x ¬Graduating(x) ∨ Happy(x)
2. ∀y ¬Happy(y) ∨ Smiling(y)
3. Graduating(NoName1)
4. ∀w ¬Smiling(w)

Step 5. Drop all ∀.

1. ¬Graduating(x) ∨ Happy(x)
2. ¬Happy(y) ∨ Smiling(y)
3. Graduating(NoName1)
4. ¬Smiling(w)

Example 4 (continued)

1. ¬Graduating(x) ∨ Happy(x)
2. ¬Happy(y) ∨ Smiling(y)
3. Graduating(NoName1)
4. ¬Smiling(w)

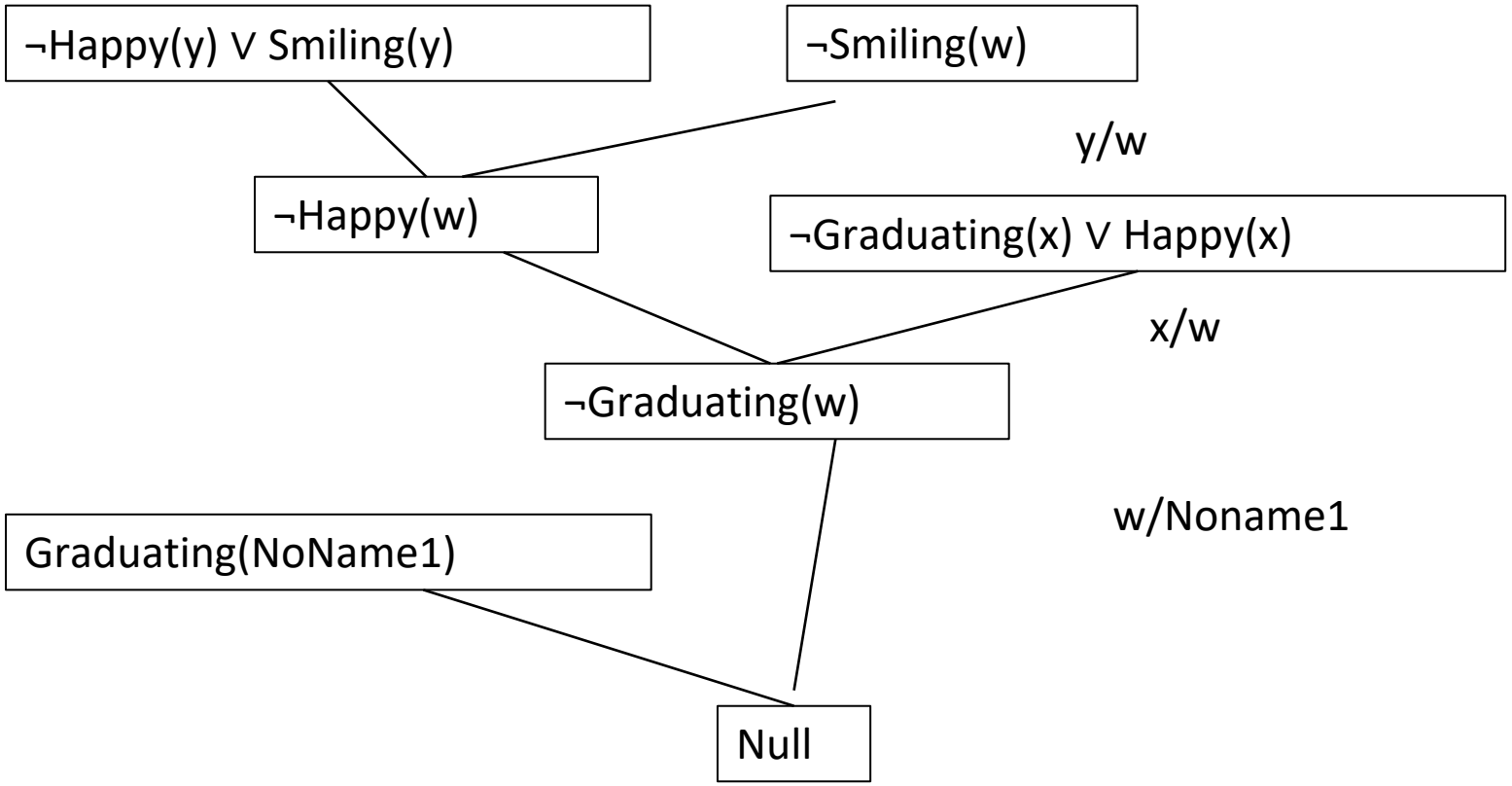
Step 6. Distribute ∧ over ∨.

(not needed)

Example 4 (continued)

1. ¬Graduating(x) V Happy(x)
2. ¬Happy(y) V Smiling(y)
3. Graduating(NoName1)

Goal : ¬Smiling(w)



Example 5

- John likes all kind of food.
- Apple and vegetable are food.
- Anything anyone eats and not killed is food.
- Bill eats peanuts and still alive.
- Harry eats everything that Anil eats.
- Goal: John likes peanuts.

Example 5 (continued) - FOL

- .John likes all kinds of food.

∀x food(x) => likes(john, x)

- Apple and vegetable are food.

food(apple) ∧ food(vegetables)

- Anything anyone eats and not killed is food.

∀x ∀y [eats(x, y) /\ ¬killed(x, y) => food(y)]

- Bill eats peanuts and still alive.

eats(bill, peanuts) /\ alive(bill)

- Harry eats everything that Anil eats.
- ∀x eats(Anil, x) => eats(Harry, x)

- Goal: John likes peanuts.

likes(John, Peanuts)

Example 5 (continued) - CNF

- **Eliminate all implication (→) and rewrite:**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀x ∀y ¬ [eats(x, y) Λ ¬ killed(x)] V food(y)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀x ¬ eats(Anil, x) V eats(Harry, x)
  - ∀x¬ [¬ killed(x)] V alive(x)
  - ∀x ¬ alive(x) V ¬ killed(x)
  - likes(John, Peanuts).

∀x: food(x) → likes(John, x)

food(Apple) Λ food(vegetables)

∀x ∀y: eats(x, y) Λ ¬ killed(x) → food(y)

eats (Anil, Peanuts) Λ alive(Anil).

∀x : eats(Anil, x) → eats(Harry, x)

∀x: ¬ killed(x) → alive(x)
∀x: alive(x) →¬ killed(x)
} **added predicates.**

likes(John, Peanuts)

Example 5 (continued) - CNF

- **Move negation (¬)inwards and rewrite**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀x ∀y ¬ eats(x, y) V killed(x) V food(y)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀x ¬ eats(Anil, x) V eats(Harry, x)
  - ∀x ¬killed(x)] V alive(x)
  - ∀x ¬ alive(x) V ¬ killed(x)
  - likes(John, Peanuts).

Example 5 (continued) - CNF

- **Rename variables or standardize variables**
  - ∀x ¬ food(x) V likes(John, x)
  - food(Apple) Λ food(vegetables)
  - ∀y ∀z ¬ eats(y, z) V killed(y) V food(z)
  - eats (Anil, Peanuts) Λ alive(Anil)
  - ∀w¬ eats(Anil, w) V eats(Harry, w)
  - ∀g ¬killed(g)] V alive(g)
  - ∀k ¬ alive(k) V ¬ killed(k)
  - likes(John, Peanuts).
- **Eliminate existential instantiation quantifier by elimination.** In this step, we will eliminate existential quantifier ∃, and this process is known as **Skolemization.**

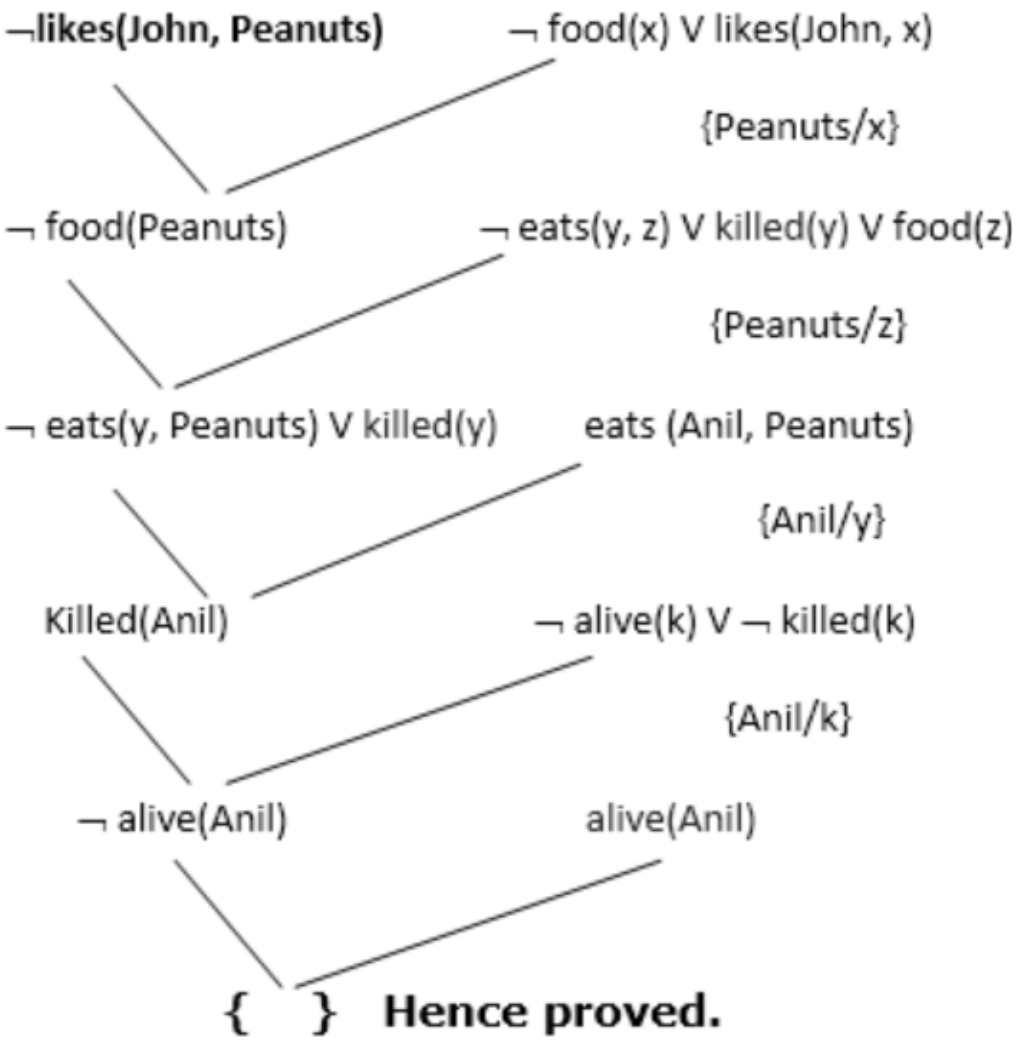
Example 5 (continued) - CNF

- **Drop Universal quantifiers.**
  In this step we will drop all universal quantifier since all the statements are not implicitly quantified so we don't need it.
  - ¬ food(x) V likes(John, x)
  - food(Apple)
  - food(vegetables)
  - ¬ eats(y, z) V killed(y) V food(z)
  - eats (Anil, Peanuts)
  - alive(Anil)
  - ¬ eats(Anil, w) V eats(Harry, w)
  - killed(g) V alive(g)
  - ¬ alive(k) V ¬ killed(k)
  - likes(John, Peanuts).

Example 5 (continued) - CNF

- **Negate the statement to be proved**
- Apply negation to the conclusion statements, which will be written as ¬likes(John, Peanuts)

Resolution Tree



Example 4

- Translate the following set of sentences into FOL. Use Forward and Backward chaining to prove if Marcus was loyal to Caesar.
  - Marcus was a man.
  - Marcus was a Pompeian.
  - All Pompeian were Romans.
  - Caesar was a ruler.
  - All Romans were either loyal to Caesar or hated him.
  - Everyone is loyal to someone.
  - People only try to assassinate rulers they aren't loyal to.
  - Marcus tried to assassinate Caesar.

Question ?